

**FULL TECHNICAL DOCUMENTATION**

AI Crop Dashboard

A complete technical reference covering every button, feature, ML model, backend architecture, and workflow of the AI-powered agricultural decision support system.

2,200

Training Samples

22

Crop Classes

~97%

Model Accuracy

60

Training Epochs

Technology Stack: **TensorFlow.js 4.20** • **Supabase** • **Chart.js** • **Vanilla JS**

Dataset: **Kaggle Crop Recommendation Dataset** • Version: **2.0**





Table of Contents

1. Project Overview

pg 3

1.1 What is AI Crop Dashboard?

1.2 Technology Stack

1.3 Project File Structure

2. Pages & User Interface

pg 5

2.1 Dashboard (index.html)

2.2 Analytics (analytics.html)

2.3 All Data (alldata.html)

3. All Buttons & Controls Explained

pg 7

3.1 Navigation Buttons

3.2 Auth Buttons (Login / Signup)

3.3 Sliders & Form Controls

3.4 Predict Crop Button

3.5 Weather Alert Buttons

4. Machine Learning Model

pg 9

4.1 Dataset — Kaggle Crop Recommendation

4.2 Neural Network Architecture

4.3 Training Process (Step by Step)

4.4 Prediction / Inference

4.5 Model Accuracy & Evaluation

5. Backend — How Supabase Works

pg 12

5.1 Supabase Overview

5.2 Authentication Flow

5.3 Database Schema

5.4 Data Read / Write Operations

5.5 Row Level Security (RLS)

6. Analytics & Charts

pg 15

6.1 Chart 1 — Submissions Over Time

6.2 Chart 2 — Crop Distribution

6.3 Chart 3 — Weather Variation

6.4 Chart 4 — State-wise Submissions

7. Weather Alert System

pg 17

8. Local Server Setup

pg 18

9. Data Flow — End to End

pg 19

10. Troubleshooting Guide

pg 20



CHAPTER 01

Project Overview

1.1 What is AI Crop Dashboard?

AI Crop Dashboard is a browser-based agricultural intelligence platform that helps farmers make data-driven decisions about which crop to grow. It combines a **real neural network trained on Kaggle data**, live charts from a PostgreSQL database, and a modern premium UI — all running in a single web browser with no separate backend server needed.



AI Prediction

TensorFlow.js neural network predicts the best crop based on 7 field parameters with ~97% accuracy.



Live Analytics

4 interactive Chart.js charts built from real Supabase database records.



Weather Alerts

6 regional weather alerts with detailed modal popups and farmer advisories.



Secure Auth

Supabase email + password authentication with protected routes.



Data Management

Farmers can submit and view field data records stored in Supabase PostgreSQL.



Premium UI

Dark mode, glassmorphism, micro-animations, gradient themes — zero frameworks.

1.2 Technology Stack

Layer	Technology	Purpose	Version
 ML Engine	TensorFlow.js	Neural network training & inference in browser	4.20.0
 Database	Supabase (PostgreSQL)	Farmer data storage + real-time queries	v2
 Auth	Supabase Auth	Email + password login & session management	v2
 Charts	Chart.js	Interactive data visualizations	4.x
 Frontend	Vanilla HTML/CSS/JS	UI, pages, interactions — no frameworks	ES2022
 Dataset	Kaggle CSV	2200 real agronomic samples for ML training	—
 Fonts	Google Fonts — Inter	Typography	—
 Server	Python http.server / Node	Local HTTP server for development	—

1.3 Project File Structure

ai-crop-dashboard/

```
|
├─ index.html      ← Dashboard home page (form + stats)
├─ analytics.html  ← AI prediction + charts + weather alerts
├─ alldata.html    ← Table of all farmer records
|
├─ Crop_recommendation.csv ← Kaggle dataset (2200 rows)
├─ schema.sql      ← Supabase database schema
├─ README.md       ← GitHub readme
|
├─ css/
|   └─ style.css    ← All styles (39KB, 1000+ lines)
|
├─ js/
|   └─ supabase-config.js ← Supabase URL + anon key
|   └─ auth.js        ← Login, signup, session logic
|   └─ data.js         ← Database read/write helpers
|   └─ dashboard.js    ← Home page stats + form submit
|   └─ analytics.js    ← TF.js ML + charts + alerts (43KB)
|   └─ alldata.js      ← Data table rendering + search
|
├─ start-server.ps1 ← PowerShell local server script
└─ run.bat          ← Windows batch server launcher
```



CHAPTER 02

Pages & User Interface

2.1 Dashboard — index.html

The main landing page. Users log in here and see their farm overview.

Section	What It Shows	Data Source
Top Navigation Bar	Logo, page links, dark mode toggle, logout button	Static HTML
Stats Cards (4 cards)	Total Submissions, Active Farmers, Avg Temperature, Top Crop	Supabase DB
Farmer Data Form	14 input fields — name, state, crop, soil, weather, nutrients, pH, irrigation, season	User Input → Supabase
Recent Activity	Last 5 farmer submissions in a list	Supabase DB

2.2 Analytics — analytics.html

The most complex page. Contains the ML prediction engine, 4 charts, and 6 weather alerts.

Section	What It Shows
ML Training Progress Panel	Blue progress bar, epoch counter, loss & accuracy live updates
Weather Alert Grid	6 clickable alert chips (Cyclone, Rainfall, Heat Wave, Frost, Good Weather, Locust)
Chart 1 — Submissions Over Time	Line chart: last 14 days of data submissions
Chart 2 — Crop Distribution	Doughnut chart: which crops farmers grow most
Chart 3 — Weather Variation	Bar + line chart: 12-month temp, humidity, rainfall
Chart 4 — State-wise Data	Horizontal bar chart: top 10 states by submissions
AI Prediction Form	7 sliders + 3 dropdowns for field conditions
Prediction Result	Crop emoji, confidence %, top-4 rankings, parameter comparison table
Supported Crops Legend	22 crop badges at the bottom

2.3 All Data — alldata.html

A full-page table showing every farmer record stored in Supabase.

Section	Details
Search Bar	Filter records by farmer name, state, or crop type
Data Table	All columns: ID, Farmer, State, Crop, Soil, Temperature, Humidity, Rainfall, N/P/K, pH, Season, Timestamp
Pagination	Navigate pages for large datasets
Export Row	Total count display

CHAPTER 03

 All Buttons & Controls Explained

3.1 Navigation Buttons

Button / Link	Location	What Happens When Clicked
 AI Crop Dashboard (Logo)	Top-left, all pages	Navigates to <code>index.html</code>
Dashboard	Nav bar	Goes to <code>index.html</code>
Analytics	Nav bar	Goes to <code>analytics.html</code>
All Data	Nav bar	Goes to <code>alldata.html</code>
 Dark Mode Toggle	Nav bar, top-right	Toggles dark/light class on <code><body></code> . Preference saved in <code>localStorage</code>
Logout	Nav bar, top-right	Calls <code>supabase.auth.signOut()</code> , redirects to login page

3.2 Auth Buttons (Login / Signup)

Button	JS Function Called	What Happens
Login	<code>handleLogin()</code> in <code>auth.js</code>	Calls <code>supabase.auth.signInWithPassword({email, password})</code> . On success → redirects to dashboard. On fail → shows error toast.
Sign Up	<code>handleSignup()</code> in <code>auth.js</code>	Calls <code>supabase.auth.signUp({email, password})</code> . Creates user in Supabase Auth. Sends confirmation email if enabled.
Show Password 	Inline toggle	Toggles <code>input[type]</code> between <code>password</code> and <code>text</code>
Switch to Sign Up / Login	Tab toggle	Shows/hides login or signup form sections without page reload

3.3 Sliders & Form Controls

The prediction form has 7 range sliders. Each slider is wired in `bindRangeInputs()` in `analytics.js` :

Slider ID	Parameter	Min	Max	Default	Unit
pTemp	Temperature	5	50	28	°C
pHumidity	Humidity	10	100	60	%
pRainfall	Rainfall	0	400	100	mm
pPh	Soil pH	3.5	9.5	6.5	—
pNitrogen	Nitrogen	0	140	50	kg/ha
pPhosphorus	Phosphorus	0	145	40	kg/ha
pPotassium	Potassium	0	205	40	kg/ha

How sliders work: Each slider fires an `input` event on every movement. The event listener simultaneously updates: (1) the live value displayed next to the slider header (`tempVal` , etc.) and (2) the input summary preview below the form (`sum_temp` , etc.).

3.4 Predict Crop Button

State	Appearance	Behavior
Loading (model training)	Grey pulsing, text: " ⌚ Training Model... Please Wait"	<code>disabled=true</code> . Clicking does nothing.
Ready	Green-orange gradient, text: " 🖨️ Predict Suitable Crop"	Enabled. Triggers form submit event.
Running inference	Disabled, text: " 🔄 Running inference..."	Temporarily disabled while TF.js computes.

On submit: reads all 7 slider values + season/soil dropdowns → normalizes → runs neural network → calls `showPredictionResult()` .

3.5 Weather Alert Buttons

Alert Chip	Severity	On Click
 Cyclone Warning	EXTREME	Opens modal with wind speed, rainfall, affected crops, 5 advisory tips
 Heavy Rainfall	HIGH	Opens modal with drainage advisory
 Heat Wave	HIGH	Opens modal with irrigation timing advisory
 Frost Risk	MODERATE	Opens modal with crop cover advisory
 Good Weather	NONE	Opens modal with sowing opportunity advisory
 Locust Alert	HIGH	Opens modal with spray/contact department advisory

The modal has a **Close (X)** button and also closes when: clicking outside the modal overlay, or pressing the **Escape** key.



CHAPTER 04

Machine Learning Model

4.1 Dataset — Kaggle Crop Recommendation

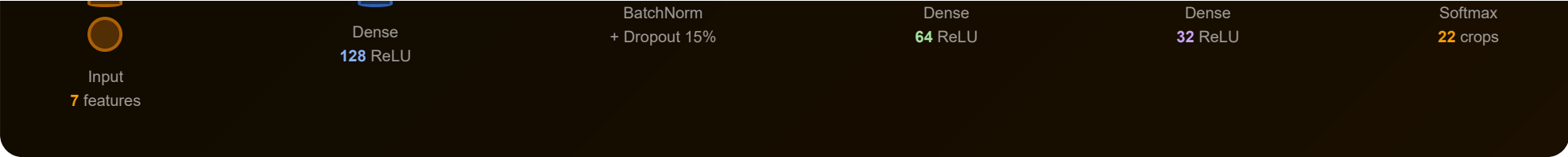
Property	Value
Source	Kaggle — Atharva Ingle (<code>Crop_recommendation.csv</code>)
Total Rows	2,200
Crop Classes	22
Rows per Crop	100 (perfectly balanced)
Input Features	N (Nitrogen), P (Phosphorus), K (Potassium), Temperature (°C), Humidity (%), pH, Rainfall (mm)
Label Column	<code>label</code> — crop name in lowercase (e.g. <code>rice</code> , <code>maize</code>)

All 22 Crop Classes

- Rice
- Maize
- Chickpea
- Kidney Beans
- Pigeon Peas
- Moth Beans
- Mung Bean
- Black Gram
- Lentil
- Pomegranate
- Banana
- Mango
- Grapes
- Watermelon
- Muskmelon
- Apple
- Orange
- Papaya
- Coconut
- Cotton
- Jute
- Coffee

4.2 Neural Network Architecture





Layer	Type	Units/Rate	Activation	Purpose
Input	Dense	128 units	ReLU	Extract high-level patterns from 7 features
Normalization	BatchNormalization	—	—	Stabilize training, faster convergence
Regularization	Dropout	15%	—	Prevent overfitting
Hidden 1	Dense	64 units	ReLU	Mid-level feature abstraction
Regularization	Dropout	10%	—	Additional regularization
Hidden 2	Dense	32 units	ReLU	Final feature compression
Output	Dense	22 units	Softmax	Probability for each of the 22 crops

4.3 Training Process — Step by Step

- 1

Page Load → TF.js CDN loaded

TensorFlow.js 4.20 is loaded from jsDelivr CDN before analytics.js runs.
- 2

fetch('Crop_recommendation.csv')

The real Kaggle CSV is fetched via HTTP from the project root. All 2200 rows are parsed into feature arrays [N,P,K,Temp,Humid,pH,Rain] and one-hot label arrays.
- 3

Min-Max Normalization

Each of the 7 features is scaled to [0, 1] using the column-wise min/max from the training data. The scaler parameters are saved for use during prediction.

4 Model Build

`tf.sequential()` creates the 7-layer network. `model.compile()` sets Adam optimizer (lr=0.003), categorical cross-entropy loss, accuracy metric.

5 model.fit() — 60 Epochs

Training runs for 60 epochs with batch size 16, 10% validation split, shuffle=true. Each epoch end fires `onEpochEnd` callback which updates the progress bar and stats in the UI.

6 Tensor Disposal

After training, input/label tensors are disposed with `.dispose()` to free GPU/CPU memory.

7 Button Enabled

The "Predict Suitable Crop" button is re-enabled and the training panel updates to show ☒ accuracy, sample count, epoch count, and timestamp.

```
// Simplified training call (from analytics.js)
await mlModel.fit(xsTensor, ysTensor, {
  epochs:      60,
  batchSize:   16,
  validationSplit: 0.1,
  shuffle:     true,
  callbacks: {
    onEpochEnd: (epoch, logs) => {
      updateTrainingProgress(epoch + 1, 60, logs.accuracy, logs.loss);
    }
  }
});
```

4.4 Prediction / Inference**1 Read slider values**

7 values are read from the HTML range inputs using `getElementById(id).value`.

2 Normalize input

Each value is scaled using the saved scaler: $(\text{value} - \text{min}) / (\text{max} - \text{min})$. This must match how training data was normalized.

3 Create tensor

`tf.tensor2d([normalizedInput])` creates a 1×7 tensor.

4 Forward pass

`model.predict(inputTensor)` runs through all 7 layers and returns a 1×22 softmax output.

5 Read probabilities

`predTensor.dataSync()` reads the 22 probability values. These are sorted to find the top-4 crops.

6 Calculate suitability

The actual input is compared to the ideal center values for the winning crop. Percentage deviation is used to compute "Field Suitability %".

7 Render result

`showPredictionResult()` injects HTML into the result container with hero card, top-4 rankings, parameter comparison, and model footer.

4.5 Model Accuracy & Evaluation

Metric	Value	Notes
Training Accuracy	~97–99%	After 60 epochs on 2200 samples
Validation Accuracy	~95–97%	10% held-out validation split
Loss Function	Categorical Cross-Entropy	Standard for multi-class classification
Optimizer	Adam (lr=0.003)	Adaptive learning rate
Fallback	Synthetic Data	If CSV fetch fails, 660 synthetic samples used



CHAPTER 05

Backend — How Supabase Works

5.1 Supabase Overview

Supabase is an open-source Firebase alternative built on PostgreSQL. It provides a **REST API, Auth service, and real-time subscriptions** — accessible directly from the browser using the Supabase JavaScript SDK. No traditional backend server (Node.js, Django, etc.) is needed.



5.2 Authentication Flow

1

User enters email + password

On login form, user fills in credentials and clicks the Login button.

2

signInWithPassword() called

`supabase.auth.signInWithPassword({email, password})` sends a POST request to the Supabase Auth endpoint.

3

Supabase returns JWT token

On success, Supabase returns an access token (JWT) + refresh token. The SDK stores these in browser `localStorage` automatically.

4

Session persists across pages

`supabase.auth.getSession()` is called on every page load. If a valid token exists, user is considered logged in.

5

Logout

`supabase.auth.signOut()` clears the token from localStorage and redirects to login.

```
// auth.js - Login
const { data, error } = await supabase.auth.signInWithPassword({
  email: emailInput.value,
  password: passwordInput.value
});
if (error) showToast('error', error.message);
else window.location.href = 'index.html';
```

5.3 Database Schema

```
-- schema.sql - Run in Supabase SQL Editor
CREATE TABLE farmer_data (
  id          UUID          PRIMARY KEY DEFAULT gen_random_uuid(),
  timestamp   TIMESTAMPTZ  DEFAULT NOW(),
  farmer_name TEXT,
  state       TEXT,
  crop_type   TEXT,
  soil_type   TEXT,
  temperature NUMERIC,
  humidity    NUMERIC,
  rainfall    NUMERIC,
  nitrogen    NUMERIC,
  phosphorus  NUMERIC,
  potassium   NUMERIC,
  ph          NUMERIC,
  irrigation  TEXT,
  season      TEXT
);

-- Protect the table
ALTER TABLE farmer_data ENABLE ROW LEVEL SECURITY;

-- Allow authenticated users full access
CREATE POLICY "Allow authenticated" ON farmer_data
  FOR ALL USING (auth.role() = 'authenticated');
```

5.4 Data Read / Write Operations

Operation	JS Call	When
Insert new record	<code>supabase.from('farmer_data').insert({...})</code>	Farmer submits the data form
Read all records	<code>supabase.from('farmer_data').select('*')</code>	All Data page loads
Read for charts	<code>supabase.from('farmer_data').select('cropType,state,timestamp')</code>	Analytics page — chart rendering
Read recent 5	<code>.select('*').order('timestamp',{ascending:false}).limit(5)</code>	Dashboard recent activity widget
Count records	<code>.select('*',{count:'exact',head:true})</code>	Stats card "Total Submissions"

5.5 Row Level Security (RLS)

RLS ensures that only **logged-in users** can read or write to `farmer_data`. Every API call automatically includes the user's JWT token in the Authorization header. Supabase validates the token server-side before allowing access.

⚠ Security Note: Never expose your `service_role` key in client-side code. Only use the `anon` (public) key — it is safe to expose and respects RLS policies.

i supabase-config.js: Set `SUPABASE_URL` and `SUPABASE_ANON_KEY` to your own project values from the Supabase dashboard under Settings → API.



CHAPTER 06

Analytics & Charts

6.1 Chart 1 — Submissions Over Time (Line Chart)

Property	Value
Chart Type	Line (area fill)
X-Axis	Last 14 days (today going back)
Y-Axis	Number of farmer submissions per day
Data Source	Supabase farmer_data → groups by timestamp date
Color	Orange (#E8810A) with 8% fill

6.2 Chart 2 — Crop Type Distribution (Doughnut)

Property	Value
Chart Type	Doughnut (60% cutout)
Segments	Top 8 crop types by count
Data Source	Supabase → groups by crop_type
Colors	8-shade amber/orange palette
Legend	Right-side, crop name labels

6.3 Chart 3 — Monthly Weather Variation (Bar + Line)

Property	Value
Chart Type	Multi-dataset: Bar (Temp + Humidity) + Line (Rainfall)
X-Axis	12 months (Jan–Dec)
Left Y-Axis	Temperature (°C) and Humidity (%)
Right Y-Axis	Rainfall (mm)
Data Source	Static agronomic averages for India (hardcoded)

6.4 Chart 4 — State-wise Submissions (Horizontal Bar)

Property	Value
Chart Type	Horizontal Bar (indexAxis: 'y')
Data	Top 10 Indian states by submission count
Data Source	Supabase → groups by state
Colors	10-shade amber gradient palette



CHAPTER 07

Weather Alert System

How Weather Alerts Work

Alerts are defined as a static JavaScript array `WEATHER_ALERTS[]` in `analytics.js`. Each object has: type, icon, title, description, region, time, severity, wind speed, rainfall, duration, advisory tips array, affected crops, temperature range, humidity.

1 `renderWeatherAlerts()`

Iterates `WEATHER_ALERTS` array and injects alert chip HTML into `#alertList` container.

2 User clicks chip → `openWeatherModal(idx)`

Creates a full modal overlay `<div>` dynamically and appends it to `document.body`.

3 Modal shows full detail

Header (icon + title + badge), description, detail rows, 4 weather stat boxes, 5-point farmer advisory list.

4 `closeWeatherModal()`

Fades the overlay (opacity 0 over 200ms) then removes it from DOM. Also cleans up the Escape key listener.

Alert	Type	Severity	Region	Advisory Points
 Cyclone Warning	critical	Extreme	Odisha, West Bengal	5 tips including harvest & evacuation
 Heavy Rainfall	warning	High	Punjab, Haryana, UP	5 tips including drainage & fertilizer delay
 Heat Wave	warning	High	Rajasthan, Gujarat	5 tips including irrigation timing & mulching
 Frost Risk	info	Moderate	Himachal, J&K, Uttarakhand	5 tips including polythene covers & irrigation
 Good Weather	success	None	Maharashtra, MP	5 tips including sowing & spraying
 Locust Alert	warning	High	Rajasthan-Gujarat border	5 tips including spraying & watch committees



CHAPTER 08

Local Server Setup

Why a Local Server is Required

⚠ **Important:** You CANNOT open the app by double-clicking `analytics.html` (the `file://` protocol). Two features require HTTP:

1. `fetch('Crop_recommendation.csv')` — browsers block `fetch()` on `file://`
2. Supabase Auth — requires an HTTP origin for token cookies

Option A — PowerShell Script (Recommended for Windows)

```
# Open PowerShell in the project folder and run:
.\start-server.ps1

# Then open browser at:
http://localhost:8080
```

Option B — Batch File

```
run.bat
# Double-click run.bat in Windows Explorer
```

Option C — Python

```
python -m http.server 8080
# Works on any OS with Python 3 installed
```

Option D — Node.js

```
npx serve . -p 8080
# Requires Node.js installed
```

Supabase Configuration

```
// js/supabase-config.js - Edit these 2 values
const SUPABASE_URL      = 'https://YOUR_PROJECT.supabase.co';
const SUPABASE_ANON_KEY = 'eyJ... (your anon key) ...';
```

Get these values from: **Supabase Dashboard** → **Settings** → **API**



CHAPTER 09

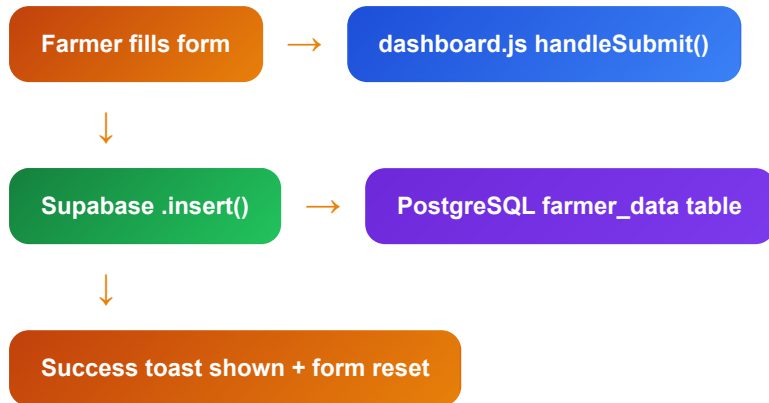
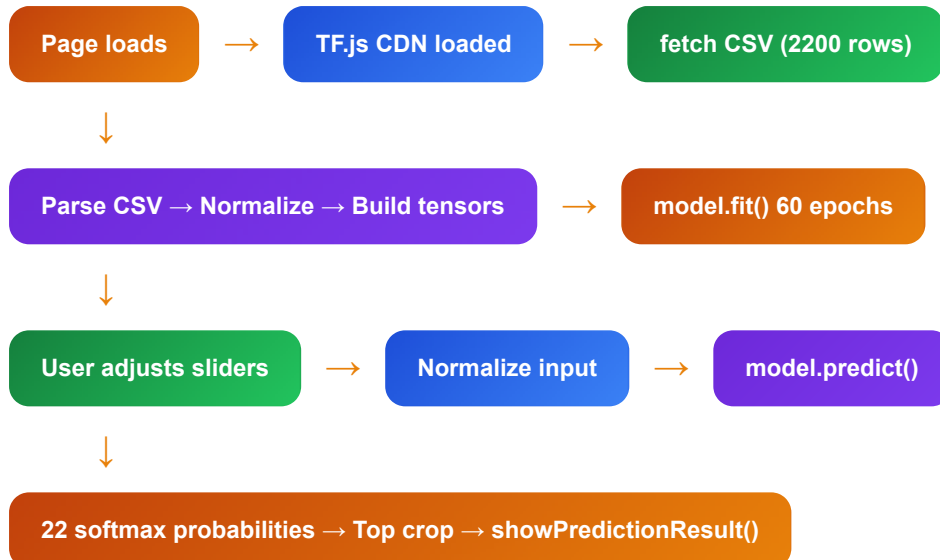
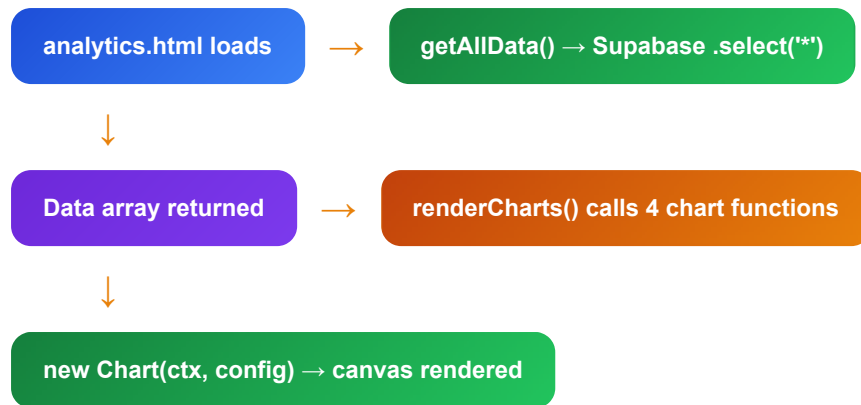
Data Flow — End to End**Farmer Submits Data****AI Prediction Flow**

Chart Data Flow





CHAPTER 10

Troubleshooting Guide

Problem	Likely Cause	Fix
Button says "Training..." and never enables	TF.js CDN failed to load, or CSV fetch failed	Check browser console for network errors. Ensure you're on <code>http://localhost</code> not <code>file://</code>
"CSV not found" toast appears	<code>Crop_recommendation.csv</code> missing from project root	Copy the CSV file to the same folder as <code>analytics.html</code>
Charts show no data	Supabase not configured or no records yet	Set correct URL + anon key in <code>supabase-config.js</code> . Add at least 1 record via the form.
"Email not confirmed" on login	Supabase email confirmation is ON	Supabase Dashboard → Auth → Settings → Disable "Enable email confirmations"
Sliders don't update live values	JS error crashing the script	Open browser DevTools (F12) → Console tab → fix any red errors shown
Prediction always shows same crop	Model not trained (mlModel is null)	Wait for training to complete. Green toast "Neural Network trained!" confirms readiness.
Page redirects to login immediately	No valid session / JWT expired	Log in again. Sessions expire after 1 hour by default in Supabase.
run.bat / start-server.ps1 doesn't work	Python or PowerShell execution policy	Run <code>python -m http.server 8080</code> manually in terminal, or use <code>npx serve . -p 8080</code>

Browser Console Debug Tips

```
// Check if TF.js loaded:
console.log(typeof tf);      // Should print "object"

// Check if model is trained:
console.log(mlModel);        // Should not be null after ~20 seconds

// Check Supabase session:
const { data } = await supabase.auth.getSession();
console.log(data.session);    // Should show user object if logged in

// Test DB connection:
const { data, error } = await supabase.from('farmer_data').select('*').limit(1);
console.log(data, error);
```

✅ **Everything working?** Open <http://localhost:8080/analytics.html> — you should see the blue training bar fill up, then a green ✅ confirmation toast, and the predict button turns orange-green. Enter your field conditions and click Predict!



AI Crop Dashboard

Full Project Documentation • Built with TensorFlow.js, Supabase & Chart.js • Kaggle Crop Dataset

Made with ❤️ for Indian Farmers